

Statistical Programming Languages

String Handling in R

Katarzyna Reluga¹

¹Acknowledgements: Thanks to Manuel Pfeuffer, Maarten Jung, and Tobias Wistub for sharing notes from earlier editions.

Strings in R

- ▶ Strings are often a natural first step to save information
 - ▶ Text and numbers combined
 - ▶ Often easiest way get data from external sources
- ▶ Strings are entered using " or '

```
string1 <- "Hello 42"  
string1  
## [1] "Hello 42"
```

```
string2 <- 'This string carries a "quote" inside itself.'  
string2  
## [1] "This string carries a \"quote\" inside itself."
```

Concatenating Strings

- ▶ `paste()` takes arbitrary number of arguments and concatenates them
- ▶ Argument `sep = "*"` specifies the character string to separate the terms (default $\rightarrow$ `sep = " "`)
- ▶ Vector arguments are recycled as needed

```
a <- "Hello"; b <- 42
paste(a, b)
## [1] "Hello 42"
paste(a, b, sep = ",")
## [1] "Hello,42"

paste0(c("X", "Y"), 1:4) # no space between X and 1
## [1] "X1" "Y2" "X3" "Y4"

c <- diag(2) # c is an identity matrix
paste(c)
## [1] "1" "0" "0" "1"
paste(c, collapse = " ")
## [1] "1 0 0 1"
```

String Manipulation

```
a <- c("Banana", "Pineapple")

# Convert characters to lower or upper case
toupper(a)
## [1] "BANANA"      "PINEAPPLE"
tolower(a)
## [1] "banana"      "pineapple"

# Substitute characters in a string
chartr(old = "a", new = "o", a)
## [1] "Bonono"      "Pineopple"

# Number of characters in a string
nchar(a)
## [1] 6 9

# Extract a substring
substr(a, start = 2, stop = 4)
## [1] "ana" "ine"
```

Regular Expression

- ▶ pattern that describes a set of strings, e.g. the pattern "ea" matches

*A thing of beauty is a joy forever.
Its loveliness increases; it will never
pass into nothingness . . .
—John Keats*

Source: *The Most Beautiful Poems in the English Language*

- ▶ types of regular expressions used in R
 - ▶ POSIX extended regular expressions (the default)
 - ▶ Perl-like regular expressions
- ▶ basic functions: `grep`, `grep1`, `regexpr`, `gregexpr`, `sub`, `gsub` and `strsplit`
- ▶ used in other functions: `apropos`, `browseEnv`, `help.search`, `list.files` and `ls`
- ▶ [Regular expression cheat sheet](#), `?regex`

Detect Pattern

- ▶ `grep` returns the index of lines where matches are found
- ▶ `grepl` returns a logical vector in which lines match are found

```
poem <- c("A thing of beauty is a joy forever.",  
          "Its loveliness increases; it will never",  
          "pass into nothingness...",  
          "--John Keats")  
  
grep("ea", poem)  
## [1] 1 2 4  
  
grepl("ea", poem)  
## [1] TRUE TRUE FALSE TRUE
```

Locate Pattern I

- ▶ `regexpr` returns the position of **ea** in the lines
 - ▶ `-1` indicates no match
- ▶ the attribute `match.length` returns the length of the match, here either 2 or `-1`
- ▶ `regexpr` identifies the first match

```
regexpr("ea", poem)
## [1] 13 20 -1 9
## attr("match.length")
## [1] 2 2 -1 2
## attr("index.type")
## [1] "chars"
## attr("useBytes")
## [1] TRUE
```

Locate Pattern II

- ▶ `gregexpr` useful if they are multiple matches in one line
- ▶ `gregexpr` returns a list for each line of text containing all match positions in a vector

```
res <- gregexpr("es", poem) # res is a 4-el. list
res[[2]] # result on line 2
## [1] 12 23
## attr(,"match.length")
## [1] 2 2
## attr(,"index.type")
## [1] "chars"
## attr(,"useBytes")
## [1] TRUE
```

- ▶ `ea` appeared once in line 2, `es` twice → `regexpr()` would only return the first match per string

Constant and Non-Constant Patterns

- ▶ If the pattern is an exact, fixed string, set `fixed = TRUE` – it runs faster

```
grep("ea", poem, fixed = TRUE)
## [1] 1 2 4
grep1("ea", poem, fixed = TRUE)
## [1] TRUE TRUE FALSE TRUE
```

- ▶ Non-constant patterns contain
 - ▶ meta characters
 - ▶ character classes
 - ▶ quantifiers
- ▶ `gregexpr("o[[:lower:]]", poem)` will match

*A thing **of** beauty is a joy forever.
Its **loveliness** increases; it will never
pass into **nothingness** ...
—John Keats*

Meta Characters: Anchors

- ▶ `^` – start of string (start of line in multiline mode)
- ▶ `$` – end of string (before final `\n`; end of line in multiline mode)

```
x <- c("cat", "scat", "catfish", "a cat", "concatenate", "fish\ncat")
```

```
# Start of string (^)
```

```
grepl("^cat", x)
```

```
## [1]  TRUE FALSE  TRUE FALSE FALSE FALSE
```

```
# End of string ($)
```

```
grepl("cat$", x)
```

```
## [1]  TRUE  TRUE FALSE  TRUE FALSE  TRUE
```

Meta Characters: Groups & alternation

- ▶ (...) – capturing group (evaluate first)
- ▶ | – alternation (“or”) between expressions

```
# Apply a quantifier to the group
# "(ha){3}" means "ha" three times in a row
a = c("hahaha", "haha", "ha ha ha")
grepl("(ha){3}", a)
## [1] TRUE FALSE FALSE

# Or expression
b = c("catfish", "hotdog", "hamster")
grepl("cat|dog", b)
## [1] TRUE TRUE FALSE
```

Meta Characters: Character Classes

- ▶ `[abc]` — any one of **a**, **b**, or **c**
- ▶ `[^abc]` — any one character **except** **a**, **b**, or **c**. `^` negates only the very first character after `[`.
- ▶ `.` — any character except a new line

```
#[abc] - any one of a, b, or c
grepl("[abc]", c("a", "bee", "cup", "dog"))
## [1] TRUE TRUE TRUE FALSE
```

```
#[^abc] - any one character except a, b, or c
grepl("[^abc]", c("a", "abc", "cab", "da", "eel"))
## [1] FALSE FALSE FALSE TRUE TRUE
```

```
#. -- any character except newline
grepl(".", c("A", "", "c"))
## [1] TRUE FALSE TRUE
```

Quantifiers

- ▶ * - Matches the preceding element zero or more times
- ▶ + - Matches the preceding element one or more times
- ▶ ? - Matches the preceding element zero or one time
- ▶ {m,n} - Matches the preceding element at least m and not more than n times

```
# * (zero or more)
# ^ha*$: ^ start of string, the letter h
# zero or more a's, $ end of string
grepl("^ha*$", c("h", "ha", "haaa", "ba"))
## [1] TRUE TRUE TRUE FALSE
# + (one or more)
grepl("^ha+$", c("h", "ha", "haaa"))
## [1] FALSE TRUE TRUE
# ? (zero or one)
grepl("^colou?r$", c("color", "colour", "colouur"))
## [1] TRUE TRUE FALSE
# {m,n} (at least m and at most n)
grepl("^a{2,4}$", c("a", "aa", "aaa", "aaaa", "aaaaa"))
## [1] FALSE TRUE TRUE TRUE FALSE
```

Meta characters II

- ▶ `regexpr("(joy|love)", poem)` will match

*A thing of beauty is a **joy** forever.
Its **loveliness** increases; it will never
pass into nothingness ...
—John Keats*

- ▶ `regexpr("i[[:lower:]]{2}", poem)` will match

*A **thing** of beauty is a joy forever.
Its lovel**iness** **increases**; it **will** never
pass **into** **nothingness** ...
—John Keats*

Meta characters III

- ▶ To match any meta character use a preceding `\`, for example `\.` matches the dot
- ▶ Further special characters
 - ▶ `\t` - tab
 - ▶ `\r` - carriage return
 - ▶ `\n` - newline
 - ▶ `\v` - vertical tab
 - ▶ `\f` - form feed
 - ▶ `\xhh` - specifies a character by two hex digits
 - ▶ `\x{h...}` - specifies a character by one or more hex digits (Unicode)
- ▶ Characters sequences with –
 - ▶ `[a-z]` matches all lowercase letters
 - ▶ `[a-f]` matches a till f

Character Classes

POSIX	Perl	ASCII	Description
[:blank:]		[\t]	Space and tab
[:cntrl:]		[\x00-\x1F\x7F]	Control characters
[:digit:]	\d	[0-9]	Digits
\	\D	[^0-9]	Non-digits
[:lower:]		[a-z]	Lowercase letters
[:punct:]		[[! " # \$ % & ' () * + , . / : ; < = > ? @ \ ^ _ { } ~]]	Punctuation characters
[:space:]	\s	[\t\r\n\v\f]	White space characters
\	\S	[^ \t\r\n\v\f]	Non-white space characters
[:upper:]		[A-Z]	Uppercase letters
[:xdigit:]		[A-Fa-f0-9]	Hexadecimal digits
[:alnum:]		[A-Za-z0-9]	Alphanumeric characters
.	.	.	
.	.	.	
.	.	.	

Example I

► Find leading and/or trailing white spaces

```
poem <- c("  A thing of beauty is a joy forever.",
          "Its loveliness increases; it will never",
          "  pass into nothingness...  ",
          "--John Keats")

# leading white space
grep("^[ \t]+", poem) # ^:beginning, (space)\t, +: 1 or more
## [1] 1 3

# trailing white space
grep("[ \t]+$", poem) # $:end
## [1] 3

grep("^[ \t]+|[ \t]+$", poem) # both white space types
## [1] 1 3
```

► **Hint:** Build a regular expression in smaller steps!

Example II

► Find HTML tags

```
poem <- c("A thing of beauty is a joy forever.",
          "Its loveliness increases; it will never ",
          "pass into nothingness...",
          "<b>--John Keats</b>")

# look for <.> and </.>
# <, . any symbol, * 0 or more, >
grep("<.*?>.*?<.*?>", poem)
## [1] 4

# The ?s are here to make the * quantifiers lazy (non-greedy)
# so they stop as soon as possible. .*? matches the shortest tag,
# e.g. <b> (without it, <.*> would gobble up to the last >).
```

► Find an e-mail address

```
email <- c("Sigbert Klinke", "sigbert@hu-berlin.de")
# allowed + (1 or more): any letter or number and symbols ._%+-
# . just a dot because inside []
# to use "\" we need to include it twice
grep("[a-zA-Z0-9._%+-]+@[a-zA-Z0-9\\.\\-]+", email)
## [1] 2
```

Extract Patterns

- ▶ `regmatches` extracts or replaces matched substrings

```
regmatches(poem, regexpr("<.*?>", poem)) # single match <.*?>
## [1] "<b>"

regmatches(poem, regexpr("<.*?>.*?<.*?>", poem))
## [1] "<b>--John Keats</b>"

res <- regmatches(poem, gregexpr("<.*?>", poem)) # multiple match <.*?>
res[3:4] # nothing in line 3, two tags in line 4
## [[1]]
## character(0)
##
## [[2]]
## [1] "<b>" "</b>"

match <- gregexpr("<.*?>.*?<.*?>", poem) # define expression first
res <- regmatches(poem, match)
res[4]
## [[1]]
## [1] "<b>--John Keats</b>"
```

Replace Patterns

- ▶ `sub` and `gsub` replace a pattern once or several times
- ▶ `\\n` is a reference to the n -th bracket pair

```
sub("<.*?>", '<i>', poem)[4] # single replace
## [1] "<i>--John Keats</b>"
# < - literal "<", .*? - minimal text up to.., > - literal

gsub("<.*?>", '<i>', poem)[4] # multiple replace
## [1] "<i>--John Keats<i>"

# find both <?.> and </.>
gsub("<(/*)..*?>", '<\\1i>', poem)[4]
## [1] "<i>--John Keats</i>"

# < - a literal "<"
# (/*) - capturing group 1: zero or more slashes `/'
# / might be here or not (`<tag>` or `</tag>`)
# .*? - any characters, as few as needed
# > - a literal ">"
# < ... > - rebuild a tag
# \\1 - insert what group 1 captured (either empty or `/'
# i - the tag name `i`
```

Split Text

- `strsplit(x, pattern)` – splits each element of `x` at every match of `pattern` (a separator you choose – string or regex) and returns a list of character vectors.

```
# limit output to 50 chars (to fit it on one slide)
options(width = 50)
strsplit(poem, "[ ;.]+") # [any symbol, all literal]
## [[1]]
## [1] "A"          "thing"      "of"         "beauty"
## [5] "is"         "a"          "joy"        "forever"
##
## [[2]]
## [1] "Its"          "loveliness" "increases"
## [4] "it"           "will"        "never"
##
## [[3]]
## [1] "pass"         "into"        "nothingness"
##
## [[4]]
## [1] "<b>--John" "Keats</b>"
# + one or several times
```

stringi & stringr

- ▶ `stringr` – simple functions for common text tasks,
- ▶ `stringi` – powerful engine used by `stringr`; the package provides R users with platform-independent functions known to Java, Perl, Python, PHP and Ruby programmers
- ▶ `stringr` cheat sheet, `RStudio addin`, `?str_view`

stringr pattern matching

Base R	stringr
func(pattern, text)	func(text, pattern)
grep	str_which (string, pattern, negate=FALSE)
grep1	str_detect (string, pattern, negate=FALSE)
sub	str_replace (string, pattern, replacement)
gsub	str_replace_all (string, pattern, replacement)

- `negate = TRUE` returns non-matching elements or information about them

```
library("stringr")
str_which(poem, "<.*?>")
## [1] 4
str_which(poem, "<.*?>", negate = TRUE)
## [1] 1 2 3
```

Pattern Matching Engines

```
# regex -- special symbols like . * + ? [ ] ... work
# some overhead.
str_which(poem, regex("ea"))
## [1] 1 2 4

# exact string -- no special symbols, fastest for simple searches
str_which(poem, fixed("ea"))
## [1] 1 2 4

# matches using language rules (Unicode collation/locale) --
# best for natural-language text (accents, ß/ss...), slowest
str_which(poem, coll("ea"))
## [1] 1 2 4
```


Detect Matches

```
# count matches
str_count(poem, "<.*?>")
## [1] 0 0 0 2
# positions of first match
str_locate(poem, "<.*?>")
##      start end
## [1,]    NA  NA
## [2,]    NA  NA
## [3,]    NA  NA
## [4,]     1   3
# positions of all matches
res <- str_locate_all(poem, "<.*?>")
res[[4]]
##      start end
## [1,]     1   3
## [2,]    16  19
```

Subset Strings I

```
# return strings where the pattern matches
str_subset(poem, "<.*?>")
## [1] "<b>--John Keats</b>"
# return the first matched pattern
str_extract(poem, "<.*?>")
## [1] NA      NA      NA      "<b>"
# return all matched patterns as list
res <- str_extract_all(poem, "<.*?>")
res[[4]]
## [1] "<b>" "</b>"
# return first match
str_match(poem, "<.*?>")
##      [,1]
## [1,] NA
## [2,] NA
## [3,] NA
## [4,] "<b>"
```

Subset Strings II

- ▶ sub patterns can be created with a capturing group (...)

```
# return with subpatterns
str_match(poem, "<.*?>(.*?)</.*?>")
##           [,1]                [,2]
## [1,] NA                NA
## [2,] NA                NA
## [3,] NA                NA
## [4,] "<b>--John Keats</b>" "--John Keats"
# returns all matched patterns as list
res <- str_match_all(poem, "<(.*?)>")
# returns a pattern and a captured group
res[[4]]
##           [,1]    [,2]
## [1,] "<b>"      "b"
## [2,] "</b>"     "/b"
```

Split Text with Regular Expression: `str_split`

```
# boundary param is either 'character', 'word',  
# 'line_break', 'sentence'  
str_split(poem, boundary("sentence"))[[1]]  
## [1] "A thing of beauty is a joy forever."  
str_split(poem, boundary("word"))[[1]]  
## [1] "A"      "thing"  "of"     "beauty"  
## [5] "is"     "a"      "joy"    "forever"  
str_split(poem, "[ ;.]+", n = 8)[c(1,4)]  
## [[1]]  
## [1] "A"      "thing"  "of"     "beauty"  
## [5] "is"     "a"      "joy"    "forever."  
##  
## [[2]]  
## [1] "<b>--John" "Keats</b>"  
# [ ;.]+ the separator pattern: any run of spaces, ; or .  
# [...] character class, + one or more  
# n = 8 return at most 8 pieces per element.
```

Split Text with Fixed Number of Splits

```
# limit the split numbers to 4
str_split_fixed(poem, " ", n = 4)
##      [,1]      [,2]      [,3]
## [1,] "A"      "thing"    "of"
## [2,] "Its"     "loveliness" "increases;"
## [3,] "pass"    "into"      "nothingness..."
## [4,] "<b>--John" "Keats</b>"  ""
##      [,4]
## [1,] "beauty is a joy forever."
## [2,] "it will never  "
## [3,] ""
## [4,] ""
```